

NPTEL
NPTEL ONLINE COURSE

REINFORCEMENT LEARNING

PAC Explanation and Naive Algo Proff
(PAC Bounds)

Prof. Balaraman Ravindran
Department of Computer Science and Engineering
Indian Institute of Technology Madras

(Refer Slide Time: 00:22)



Chernoff Hoeffding bound, we saw that already.

Chernoff – Hoeffding Bound: Let $X_1, X_2, X_3, \dots, X_n$ be random variables with common range $[0, 1]$ such that $E[X_t | X_1, X_2, \dots, X_{t-1}] = \mu$. Let $S_n = \frac{X_1 + X_2 + \dots + X_n}{n}$. Then for $\epsilon > 0$,

$$P[S_n \geq \mu + \epsilon] \leq e^{-2\epsilon^2 n}$$

$$P[S_n \leq \mu - \epsilon] \leq e^{-2\epsilon^2 n}$$

So that is one result right we looked at it in the last class and you saw how we used it to manage to bound the regret of UCB right so we will be looking at yet another class of bandit algorithms today right so the PAC bandits and we will be using the Chernoff hoeffding bound again to bound a slightly different quantity but nevertheless of interest.

Markov Inequality:

$$Pr(X \geq a) \leq \frac{E[X]}{a}; a > 0$$

you remember the Markov inequality Elec people, people who did probability theory one form of writing it is this which is what we will need right assuming a is, a is positive, the expected.

So, the x support is in the positive range I mean so I mean whatever makes appropriate sense for you to make sure that this is a probability right

Union Bound:

$$P(\cup_i A_i) \leq \sum_i P(A_i)$$

next thing you use is the Union bound this is this all of you should know the probability of the union of multiple events right is less than or equal to the sum of the probabilities of the individual events that is rather straightforward it has a fancy name called Union bound right so whenever somebody says oh, hence by Union bound we can write this result essentially all they are saying is that they are upper bounding the probability of the joint event by the sum of the probabilities of the individual events okay these are the results we will just keep using them as we will do some of the proofs today.

Okay so we talked about multiple solution concepts for bandit algorithms right so the first one was asymptotic correctness and the second one we talked about was regret optimality which we already looked at one example of a regret optimal algorithm UCB or UCB1 more correctly even though popularly when people say UCB they refer to UCB1 right and then what is the third one PAC optimality right which was the probably approximately correct thing where you are trying to bound

with some probability right that I mean you are close enough to the true solution right So that is essentially the guarantee that we want to give right.

(Refer Slide Time: 05:20)



So what we look at today are right so PAC bounds for multi-armed bandit problems right I will start off with a very, very simple algorithm start off with a very, very simple algorithm right this is just to get you started right it is not a true algorithm that you have to use, its just to get you started so I have an input which is I will give you an ϵ and a δ and what I am asking for is a guarantee that at the end of this algorithm the arm that I return to you will be ϵ close to the optimal arm with probability $1 - \delta$.

Right with a very high probability if when this algorithm finishes running with a very high probability I will give you an arm which is ϵ close to the true arm the payoff of the arm that I give you will be ϵ close to the true arm so if you remember the PAC guarantee that we wrote right so most PAC algorithm PAC style algorithms essentially take this ϵ and δ as inputs right and they determine some parameters of the algorithms based on this ϵ and δ okay so I am writing the

algorithm in the same style as it is in the paper that we will give you to read so that it is easy for you to map it.

PAC Bounds for MAB

Naive Algorithm:

Input: $\epsilon > 0, \delta > 0$

Output: 'An arm'

For each $a \in A$, do

Sample 'a' l times where $l = \frac{2}{\epsilon^2} \ln\left(\frac{2K}{\delta}\right)$

Let $Q(a)$ be the average reward of arm a

End

Output $a = \operatorname{argmax}_{a \in A} Q(a)$

Unfortunately I will still do the online translation of notation so that you can compare across multiple algorithms okay, and this paper is way, way better written than the UCB paper okay so it is actually you can understand the paper when you read it okay so this is the on the median elimination algorithm by Eyal Even-Dar, Shie Mannor and Yishay Mansour okay, it is titled Action elimination for Reinforcement Learning okay so the nice thing about this ok I will tell you the nice things about the paper later.

But the really nice thing about the paper is it is well written so you can easily read it. very simple algorithm right the algorithm is a very simple algorithm what is the n in the log, where where, what is n inside the log, n is the number of arms right so what did I say I changed it to K right sorry yeah you are right over in the last class, last lecture I changed it to K yeah right this is a very simple algorithm all I am saying is take each arm sample it some number of times okay we can think about it right this is this will work see if I if I get a good enough estimate for each arm right if you get a good enough estimate of the reward of each arm then I can say that with a very high probability I will be epsilon optimal right.

So this of course there is always a small chance that I will go wrong right do you see what I am saying here so the idea is very simple so I will take an arm keep pulling it many many times right until I have gotten a good estimate of the of the expectation right so remember the probability with

which it is going to come up with 1 or whatever right so whatever is my expectation parameter I will just keep pulling the arm multiple times until I get a good estimate of the expectation I will do that for each of the arms that I have right.

And once I have a good enough estimate of the expectation all I have to do is now go and play the arm that has the highest expectation right that is essentially what I am doing here now the magic here is in that number which I have written down there right I mean given K , ϵ and δ that is actually a number okay remember I take ϵ and everything as inputs given all of this is actually a it is actually a number right so that magic number how did I produce somewhere yeah I wrote all of this I wrote all of these things so that you know that you are going to use them to produce that magic number so it is very simple.

(Refer Slide Time: 11:13)

$(\epsilon, \delta) \text{ Naive } (\epsilon, \delta) \text{ PAC}$
 arm sample complexity:
 $n(K, \epsilon, \delta)$
 for arm i : $q_{k,i}(t) \leq q_i^*(t) - \epsilon$
 $P(q_i(t) > q_i^*(t)) \leq P(q_i(t) > q_i^*(t) + \epsilon/2 \text{ OR } q_i(t) < q_i^*(t) - \epsilon/2)$
 $\leq P(q_i(t) > q_i^*(t) + \epsilon/2) + P(q_i(t) < q_i^*(t) - \epsilon/2)$
 $\leq 2e^{-\frac{\epsilon^2}{4}}$
 $n(K, \epsilon, \delta) \leq \frac{8K}{\epsilon^2 \delta}$

Summing over all i :
 $P(\text{any failure}) \leq K \cdot e^{-\frac{\epsilon^2}{4}} \leq \delta$

So let us make things a little formal so we call this algorithm naive, so then algorithm Naive which takes as input ϵ δ is an (ϵ, δ) PAC algorithm, you remember what is (ϵ, δ) PAC, we defined it in the earlier lecture right (ϵ, δ) PAC gives you that whatever guarantees you are talking about right (ϵ, δ) PAC algorithm with arm sample complexity essentially the number of times you pull the

arms given by $O\left(\left(\frac{K}{\epsilon^2}\right) \log\left(\frac{K}{\delta}\right)\right)$ right so this part is obvious right I pull each arm $\frac{1}{\epsilon^2} \ln \frac{K}{\delta}$ right so for n arms I have to do that n times so my overall sample complexity is $O\left(\left(\frac{K}{\epsilon^2}\right) \log\left(\frac{K}{\delta}\right)\right)$ okay, so the interesting part now is to show that it is actually (ϵ, δ) PAC, what is it obviously yeah for today's lecture if I write n by mistake right please point it out to me.

Theorem 1: Naive (ϵ, δ) is an (ϵ, δ) - PAC algorithm with arm sample complexity $O\left(\left(\frac{K}{\epsilon^2}\right) \log\left(\frac{K}{\delta}\right)\right)$.

Proof:

Let a' be an arm such that $q_*(a') < q_*(a^*) - \epsilon$.

$$\begin{aligned} P(Q(a') > Q(a^*)) &\leq P(Q(a') > q_*(a') + \epsilon/2 \text{ OR } Q(a^*) < q_*(a^*) - \epsilon/2) \\ &\leq P(Q(a') > q_*(a') + \epsilon/2) + P(Q(a^*) < q_*(a^*) - \epsilon/2) \\ &\leq e^{-2\frac{\epsilon^2}{4}l} * 2 \end{aligned}$$

Plug in $l = \frac{2}{\epsilon^2} \ln\left(\frac{2K}{\delta}\right)$

$$\begin{aligned} P(Q(a') > Q(a^*)) &\leq 2e^{-\frac{1}{2}\epsilon^2 \times \frac{2}{\epsilon^2} \ln\left(\frac{2K}{\delta}\right)} \\ &\leq 2e^{-\ln\left(\frac{2K}{\delta}\right)} \\ &\leq 2 \times \left(\frac{2K^{-1}}{\delta}\right) \\ &\leq \frac{\delta}{K} \end{aligned}$$

So that's just for 1 a' . Now summing over all a' ,

$$\text{Prob. Of failure} \leq (K - 1) \frac{\delta}{K} \leq \delta$$

So I will correct it to K right okay so how do we prove it well the sample complexity part is obvious we do not have to prove that we just have to prove the (ϵ, δ) PAC so for that we will assume that let a' be an arm such that $q_*(a')$ okay so a' is an arm which is not ϵ optimal right, an arm that is ϵ optimal is ϵ close to the best arm right so $q_*(a^*)$ is the reward corresponding to the best arm we know that right so I am saying that I will pick an arm a' such that it is at least ϵ away from $q_*(a^*)$ right.

So the $q^*(a')$ is at least ϵ away from $q^*(a^*)$ right so that means it is a bad arm right so arms that are more than ϵ close to the true arm I do not care right if I select them I am happy because my guarantee is satisfied I really have to worry about making a mistake which in this case is selecting an arm that is more than ϵ away from the true optimal okay so basically we are looking at the event right so this is what we want we are looking at right so if this is the case, ie. $Q(a') > Q(a^*)$ then the algorithm will output a' instead of a^* correct.

People onboard with me on this right so if $Q(a') > Q(a^*)$, then my algorithm will output at least it will not output a^* for me to select a' as the best arm at least this condition has to be satisfied okay let us put it that way right for my algorithm to output a' as the best arm for sure this condition must be satisfied in fact a' has to be higher than every other arm right only then I will output that but at least this condition has to be satisfied so essentially I am going to look at the probability of this happening $P(Q(a') > Q(a^*))$ all right and then we will try to bound that right.

So people can see why we need this condition right so if a' has to be reported as the best arm it has to be at least better than a^* right great so that is the condition so already we are being little loose here okay, this is not really the probability that yes a' will be the best arm okay but it is only looking at some sub event of that it is not all the events together but this will give me at least a bound on the probability so that is essentially why you are looking at this event okay.

Right so what is happening here, let us say that is my that is my $q^*(a^*)$ okay that is my right and this gap is at least this gap is at least ϵ , okay so there is this is the setup that we have now so and I am making some estimate of $q^*(a')$ which is $Q(a')$ right and I am making some estimate of $q^*(a^*)$ which is $Q(a^*)$ so the condition I am asking is that $Q(a')$ should be higher than I mean $Q(a^*)$ should be higher than $Q(a^*)$ right.

So essentially I am saying that I should have $Q(a')$ here $Q(a^*)$ somewhere lower right so this is the case I am looking at right so this all automatically means that either $Q(a')$ has to be more than $\epsilon/2$ from $q^*(a')$ right it should be at least ϵ more than $\epsilon/2$ above or $Q(a^*)$ should be more than $\epsilon/2$

below $q^*(a^*)$, right if they are exactly $\epsilon/2$ they will be equal I mean they won't be one above the other right if one is $\epsilon/2$ below other is $\epsilon/2$ above so you can think of cases where this is actually here right so this could be here in which case q^* I mean $q^*(a^*)$ has to be even below right it will be more than ϵ away forget about $\epsilon/2$ right so likewise for cases where you are overestimating $Q(a^*)$

Then this will also have to be higher than that in which case again it will be more than ϵ away right so the interesting case is when it is in between the two right then it has to at least one of them has to be $\epsilon/2$ away from the true estimate right either as a overestimate or as an underestimate right is it clear so why I wrote this down, right one of these two events has to be satisfied. right I am just taking them as independent events and summing them up and that still be an upper bound on the total probability you have right, making things simpler and simpler for me and now what happens here it kind of in the Chernoff space right.

So I can apply Chernoff bound here so what would be the case here $e^{-2\frac{\epsilon^2}{4}l}$, yeah so let us do that so this will be $\leq e^{-2\frac{\epsilon^2}{4}l}$, what about the other term, the same right it is a symmetrical bound so if that is $\epsilon/2$ or $-\epsilon/2$, it is going to be the same right so is it clear okay so now plug in this l there so what do you get, hmm, so essentially what we need to do now is adjust this l so that all those things get cancelled out that is basically what we are looking for right and you will be left with, what should you be left with.

What should you be left with what do you think you should be left with no no no I want this probability δ this is the probability of me making a mistake right so it should be δ , should I be left with δ , not quite this is the probability of me choosing one non optimal arm by mistake what if all my arms are non-optimal except the one optimal arm right everything is away from that is the worst case so I should have δ/K or $K-1$ so we will take δ/K , okay that is easier than and so this whole thing should simplify to δ/K right so that is what I should be looking for so what should be my l right.

So this if we plug this in what will I get so the $\frac{4}{\epsilon^2}$ will go so I will get $2 \ln\left(\frac{2K}{\delta}\right)$, that is not quite right and what do I get then, yeah $2 \ln\left(\frac{2K}{\delta}\right)$ so I will get $\frac{\delta}{K}$, I will get $\frac{\delta}{K}$ is it fine I simplify this I get $\frac{\delta}{K}$ so now you know how you actually come up with this magic number 1 right, by actually deriving this bound and then figuring out what else should I plug in here so that I get the necessary error for the probability okay so that is just for one α hmm there is a 2 outside right, 2ϵ okay.

So this is clear, this is a naive algorithm and the analysis also is pretty simple, pretty straightforward right so all of you are happy with this right so if you if I give you an algorithm with a slightly different way of selecting actions you can all derive the complexity the $\ln$ that is required basically great it turns out that so we have produced this complexity here right it turns out that it is almost impossible to get rid of this $\frac{1}{\epsilon^2}$ right and $\log\left(\frac{1}{\delta}\right)$ right, so when I try to come up with another PAC algorithm right I want to you remember what I said about PAC analysis.

So I will give you an $\epsilon \delta$ and now the interesting part is telling me how you can achieve this $\epsilon \delta$ with as few samples as possible correct so in this case I am telling you that this is the sample complexity right and K figures in two places you have a $\frac{K}{\epsilon^2}$ and $\log\left(\frac{K}{\delta}\right)$ right it turns out that this $\frac{1}{\epsilon^2}$ and $\log\left(\frac{1}{\delta}\right)$ is very hard to get rid of right but then people have come up with and can you get rid of this K , it should depend on K why at least once right.

I have to try every α at least one so I cannot get rid of the K also right so the only thing I can try and minimize here is well I can try and get rid of this K right so I can try to get rid of the $\log K$ dependency right and then have $\frac{K}{\epsilon^2} \log\left(\frac{1}{\delta}\right)$ roughly I can try to move in that direction and once I reach there I will probably be in, left with trying to get rid of this two here right and try to make the 2 into $3/2$ or 1.77 or something like that. you know how these things proceed right so you can try to optimize the constants but more or less this $\frac{1}{\epsilon^2}$ and $\log\left(\frac{1}{\delta}\right)$ is something that will have to live with okay.

IIT Madras Production

Funded by
Department of Higher Education
Ministry of Human Resource Development
Government of India

www.nptel.ac.in

Copyrights reserved